

FLIPPY

Documento de Proyecto

Especificación Funcional y Técnica

Cliente
Virgilio
Desarrollador
Ezequiel Gutiérrez — Botizar
Versión
2.0 — Arquitectura Frontend/Backend separada
Fecha
Junio 2026
Presupuesto total
USD 4.300 (incl. fee Workana + módulo multimodal)
Plazo total
6 semanas y media

Índice

1. Resumen ejecutivo
2. Alcance funcional
3. Arquitectura técnica
4. Base de datos — ERD y esquema
5. Pipeline RAG
6. Módulo de pagos — Mercado Pago
7. Plan de trabajo por hitos
8. Stack tecnológico
9. Requerimientos del cliente
10. Criterios de aceptación

1. Resumen ejecutivo

Flippy es una Progressive Web App (PWA) que funciona como asistente conversacional de inteligencia artificial para la comunidad educativa inmobiliaria de Virgilio. El asistente responde consultas exclusivamente basándose en un corpus de documentos internos mediante un sistema de Recuperación Aumentada por Generación (RAG), sin mostrar citas directas al usuario.

El sistema opera bajo un modelo de suscripción mensual recurrente integrado con Mercado Pago, con dos planes de acceso: gratuito (funcionalidades limitadas) y de pago (acceso completo). La aplicación es instalable como PWA nativa en iOS y Android.

2. Alcance funcional

2.1 Módulo de chat — Flippy

- Interfaz conversacional con identidad visual de marca (paleta marfil / vino / carbón, tipografías Cormorant Garamond + Lato)
- Historial persistente por usuario: cada conversación se guarda en base de datos
- Múltiples chats en paralelo al estilo ChatGPT: sidebar con lista de conversaciones, nuevo chat, renombrar
- Respuestas ancladas exclusivamente en el corpus documental, sin citas directas visibles
- Soporte multimodal: el usuario puede adjuntar imágenes desde cámara o galería; Flippy las analiza combinándolas con el material del corpus
- Pantalla de onboarding con instrucciones de instalación en iOS (Safari → Compartir → Agregar a pantalla de inicio)

2.2 Panel de administración de documentos

- Interfaz autogestionable: subir, listar y borrar documentos sin intervención de código
- Formatos soportados: PDF, Word (.docx), texto plano (.txt), imágenes (.jpg, .png)
- Sin límite en la cantidad de fuentes
- Procesamiento automático al subir: fragmentación, generación de embeddings e indexación en pgvector

2.3 Módulo de usuarios y autenticación

- Registro e inicio de sesión propio (email + contraseña)
- Dos planes de acceso: gratuito y de pago
- Control estricto de permisos según plan activo
- Estados de cuenta: activo, en_mora, gratuito, cancelado

2.4 Módulo de suscripciones — Mercado Pago

- Alta de suscripción mensual recurrente vía checkout de MP

- Renovaciones automáticas procesadas vía webhooks
- Lógica de pagos rechazados con período de gracia (estado en_mora)
- Reintentos automáticos de MP: 4 intentos espaciados
- Degradación automática a plan gratuito al agotar reintentos
- Banner de aviso en la PWA durante el estado en_mora
- Flujo de re-suscripción para usuarios dados de baja

2.5 PWA — instalación nativa

- Instalable en iPhone (iOS vía Safari) y Android (Chrome)
- Service worker para funcionamiento offline básico
- Manifest con íconos y configuración de pantalla completa
- Pruebas obligatorias en dispositivo físico iOS, Android y Chrome desktop antes de cada hito

3. Arquitectura técnica

La arquitectura está desacoplada en dos servicios independientes: un frontend Next.js dedicado exclusivamente a la interfaz, y un backend FastAPI (Python) que concentra toda la lógica de negocio. Ambos servicios se despliegan en Railway dentro del mismo proyecto, compartiendo dashboard y gestión de variables de entorno.

3.1 Diagrama de capas

Capa	Servicio	Responsabilidad
Cliente (PWA)	Next.js 14 (App Router) en Railway	UI, service worker, manifest, instalación iOS/Android
Backend (API Core)	FastAPI (Python 3.11+) en Railway	Auth, chat RAG, ingesta de documentos, webhooks MP, streaming SSE
Base de datos	Supabase (PostgreSQL)	Usuarios, chats, mensajes, documentos, suscripciones
Vectores	Supabase pgvector	Embeddings de chunks para búsqueda semántica — sin servicio externo
Storage	Supabase Storage	Archivos originales (PDF, Word, imágenes, texto)
IA — chat	Anthropic Claude 3.5 Sonnet	Generación de respuestas RAG + análisis de imágenes
IA — embeddings	OpenAI text-embedding-3-small	Vectorización de chunks y queries
Pagos	Mercado Pago	Suscripciones recurrentes, webhooks transaccionales
Hosting	Railway	Dos servicios en un mismo proyecto, deploy continuo desde GitHub, SSL

Capa	Servicio	Responsabilidad
		automático

3.2 Comunicación frontend–backend

El frontend consume el backend exclusivamente vía fetch nativo del navegador, sin librerías externas (sin axios). Todas las llamadas están centralizadas en un único archivo de servicio del lado del cliente.

Elemento	Detalle
URL base	https://api-flippy.botizar.com/api/v1
Autenticación	Authorization: Bearer <JWT>
Sesión	Login devuelve access + refresh token, guardados en localStorage
Streaming	Server-Sent Events (SSE) leído vía ReadableStream del body de fetch
Multi-tenancy	No aplica — un solo cliente, sin headers de tenant ni API key estática

3.3 Flujo de una consulta

El ciclo de vida de una consulta de usuario sigue estos pasos en orden:

1. El usuario escribe una consulta (texto) o adjunta una imagen en la PWA (Next.js)
2. El frontend envía la solicitud al backend FastAPI vía fetch con SSE habilitado
3. FastAPI genera un embedding de la consulta usando OpenAI text-embedding-3-small
4. pgvector (en Supabase) realiza búsqueda de similitud coseno y devuelve los N chunks más relevantes del corpus
5. FastAPI construye el prompt: contexto RAG + historial de la conversación + consulta del usuario (+ imagen si aplica)
6. Claude 3.5 Sonnet procesa el prompt y devuelve la respuesta en streaming hacia FastAPI
7. FastAPI retransmite el streaming al frontend vía SSE y guarda la interacción en Supabase en paralelo

4. Base de datos — esquema

4.1 Tablas principales

users

Campo	Tipo	Descripción
id	uuid PK	Identificador único (Supabase Auth)
email	text unique	Email del usuario

Campo	Tipo	Descripción
plan	enum	gratuito pago
status	enum	activo en_mora cancelado
mp_subscription_id	text	ID de suscripción en Mercado Pago
mp_customer_id	text	ID de cliente en Mercado Pago
created_at	timestampz	Fecha de registro
updated_at	timestampz	Última actualización

chats

Campo	Tipo	Descripción
id	uuid PK	Identificador único del chat
user_id	uuid FK → users	Dueño del chat
title	text	Título editable (generado automáticamente al inicio)
created_at	timestampz	Fecha de creación
updated_at	timestampz	Última actividad

messages

Campo	Tipo	Descripción
id	uuid PK	Identificador único del mensaje
chat_id	uuid FK → chats	Chat al que pertenece
role	enum	user assistant
content	text	Contenido del mensaje
image_url	text nullable	URL de imagen adjunta en Supabase Storage
created_at	timestampz	Timestamp del mensaje

documents

Campo	Tipo	Descripción
id	uuid PK	Identificador único del documento
name	text	Nombre del archivo
type	enum	pdf docx txt image
storage_path	text	Ruta en Supabase Storage
status	enum	processing ready error
chunk_count	integer	Cantidad de chunks generados

Campo	Tipo	Descripción
created_at	timestampz	Fecha de subida

document_chunks

Campo	Tipo	Descripción
id	uuid PK	Identificador único del chunk
document_id	uuid FK → documents	Documento origen
content	text	Texto del chunk
embedding	vector(1536)	Embedding generado por OpenAI
chunk_index	integer	Posición del chunk en el documento
metadata	jsonb	Página, sección u otros metadatos

subscriptions

Campo	Tipo	Descripción
id	uuid PK	Identificador interno
user_id	uuid FK → users	Usuario suscriptor
mp_subscription_id	text	ID en Mercado Pago
status	enum	authorized paused cancelled
next_payment_date	date	Próximo intento de cobro
last_event	text	Último evento de webhook recibido
created_at	timestampz	Fecha de alta
updated_at	timestampz	Última actualización

5. Pipeline RAG

5.1 Ingesta de documentos

Cuando el administrador sube un archivo al panel, FastAPI lo guarda en Supabase Storage, responde de inmediato al cliente, y dispara el procesamiento como tarea en segundo plano (background task):

- PDF: extracción de texto con pdfplumber, preservando estructura de tablas y saltos lógicos; chunks de ~500 tokens con 50 tokens de overlap
- Word (.docx): conversión a texto con python-docx, mismo proceso de chunking
- Texto plano: chunking directo sin conversión
- Imágenes: almacenadas en Supabase Storage; Claude las procesa en tiempo de consulta (no se vectorizan)

Cada chunk pasa por OpenAI text-embedding-3-small para generar un vector de 1536 dimensiones, que se almacena en document_chunks con indexado IVFFlat en pgvector para búsqueda eficiente. No se utiliza una base de datos vectorial externa: a un volumen de 250 MB de corpus inicial, pgvector cubre la performance necesaria sin sumar otro servicio a mantener.

5.2 Estrategia de chunking

Parámetro	Valor	Justificación
Tamaño del chunk	500 tokens	Equilibrio entre contexto y precisión de recuperación, con corte inteligente por párrafo
Overlap	50 tokens	Evita cortar conceptos en los bordes
Top-K recuperados	5 chunks	Suficiente contexto sin saturar el prompt
Modelo de embedding	text-embedding-3-small	Relación costo/calidad óptima para este uso
Dimensiones del vector	1536	Default del modelo seleccionado
Índice pgvector	IVFFlat	Adecuado para corpus de 250 MB, sin necesidad de servicio externo
Orquestación	LangChain (Python)	Reutiliza patrones ya probados en producción

5.3 Construcción del prompt

El prompt enviado a Claude tiene la siguiente estructura fija:

```
SYSTEM: Eres Flippy, asistente de la comunidad inmobiliaria de [cliente]. Responde ÚNICAMENTE basándote en el contexto provisto. No menciones las fuentes. Si no encontrás la respuesta en el contexto, decilo claramente.
CONTEXT: [chunks recuperados de pgvector]
HISTORY: [últimos N mensajes del chat activo]
USER: [consulta + imagen si aplica]
```

6. Módulo de pagos — Mercado Pago

6.1 Estados de usuario

Estado	Descripción	Acceso a IA
activo	Suscripción vigente y al día	Completo
en_mora	Pago rechazado, reintentos en curso	Restringido (límite plan gratuito)
gratuito	Plan gratuito o sin suscripción	Limitado (a definir con cliente)

Estado	Descripción	Acceso a IA
cancelado	Suscripción cancelada definitivamente	Solo plan gratuito

6.2 Flujo de webhooks

El backend expone un endpoint POST `/api/webhooks/mercadopago` que procesa los siguientes eventos:

- `subscription.authorized` → cambia status a activo, plan a pago
- `invoice.paid` → confirma renovación mensual, registra en subscriptions
- `invoice.payment_failed` → cambia status a `en_mora`, restringe acceso a IA, envía notificación al usuario
- `invoice.retryed` (exitoso) → restaura status a activo
- `subscription.cancelled` → cambia status a cancelado, plan a gratuito

6.3 Lógica de mora y reintentos

Mercado Pago realiza hasta 4 reintentos automáticos ante un pago rechazado. Durante ese período:

- El usuario permanece en la app pero con acceso restringido (estado `en_mora`)
- Se muestra un banner persistente invitando a actualizar el método de pago
- Si un reintento es exitoso: el estado vuelve a activo de forma inmediata vía webhook
- Si se agotan los 4 reintentos: MP cancela la suscripción, el backend pasa el usuario a cancelado / plan gratuito
- Para reactivar: el usuario debe iniciar un nuevo flujo de suscripción con tarjeta válida

7. Plan de trabajo por hitos

Hito	Alcance	Tiempo	Monto
Hito 1 — Base Frontend + Backend	Entorno Next.js y FastAPI desplegados como servicios separados en Railway, estructura PWA (service worker, manifest, íconos iOS/Android), interfaz de chat maquetada según prototipo, sidebar de historial, aprovisionamiento Supabase + pgvector	2 semanas	USD 950 (25%)
Hito 2 — RAG + Admin + Multimodal	Panel de administración de documentos, pipeline de ingesta en Python (PDF/Word/texto), embeddings y búsqueda vectorial en pgvector, integración Claude para respuestas ancladas con streaming SSE, módulo de análisis de imágenes del usuario	3 semanas	USD 1.630 (38%)

Hito	Alcance	Tiempo	Monto
Hito 3 — Usuarios + Pagos	Sistema de autenticación email/password (JWT), control de planes y permisos, integración completa de suscripciones recurrentes MP, lógica de webhooks en FastAPI, estados de mora y baja	1 semana	USD 1.075 (25%)
Hito 4 — QA + Deploy	Pruebas en dispositivos reales (iPhone, Android, Chrome), simulación de flujos de pago en sandbox, optimización mobile, transferencia de repositorios y cuentas al cliente	1 semana	USD 645 (15%)

Total: USD 4.300 · Plazo: 6 semanas y media · Pagos por hito vía garantía Workana

8. Stack tecnológico

Arquitectura desacoplada: frontend y backend como servicios independientes, desplegados juntos en el mismo proyecto de Railway.

Categoría	Tecnología	Versión / Plan
Framework frontend	Next.js (App Router)	14.x
Lenguaje frontend	TypeScript	5.x
Framework backend	FastAPI	Python 3.11+
Hosting	Railway (2 servicios)	Starter / Pro según tráfico
Base de datos	Supabase (PostgreSQL + pgvector)	Pro plan recomendado
Storage de archivos	Supabase Storage	Incluido en plan Supabase
IA — chat y visión	Anthropic Claude 3.5 Sonnet	API pago por token
IA — embeddings	OpenAI text-embedding-3-small	API pago por token
Orquestación RAG	LangChain (Python)	0.x (pip)
Parseo PDF	pdfplumber	pip
Parseo Word	python-docx	pip
Streaming	Server-Sent Events (SSE)	Nativo de FastAPI
Pagos	Mercado Pago	API suscripciones recurrentes
Control de versiones	GitHub	2 repositorios privados del cliente (web + api)

8.1 Costos operativos estimados (mensual)

Servicio	Costo estimado / mes	Notas
Railway (frontend)	USD 5–10	Servicio Next.js
Railway (backend)	USD 5–15	Servicio FastAPI, según tráfico y uso de CPU
Supabase	USD 0–25	Free tier hasta 500 MB DB; Pro desde USD 25 — incluye pgvector
Anthropic (Claude)	~USD 0.003 por mensaje	Variable según volumen de usuarios
OpenAI (embeddings)	< USD 1	Solo al subir documentos, no por consulta
Mercado Pago	% por transacción	Sin costo fijo mensual
Total infraestructura base	USD 15–55 / mes	Sin costos de IA variables; sin servicios vectoriales externos

9. Requerimientos del cliente

Los siguientes puntos deben estar resueltos antes del inicio del Hito 1:

- **Dominio propio comprado y disponible (necesario para configurar service worker en iOS)**
- **Cuenta de Mercado Pago con módulo de suscripciones recurrentes habilitado — requiere solicitud a MP, puede tardar días hábiles**
- **Cuenta de Anthropic con billing activo y tarjeta cargada — acceso vía invitación de colaborador**
- **Definición del plan gratuito: límite de mensajes por día/mes o acceso a corpus reducido (a confirmar con Virgilio)**

Accesos a entregar al desarrollador como colaborador (sin compartir contraseñas):

- Supabase: invitación de colaborador al proyecto
- Anthropic: acceso a la organización
- Railway: invitación al equipo del proyecto
- GitHub: acceso al repositorio privado

10. Criterios de aceptación

Criterio	Descripción	Hito
Pruebas en dispositivos reales	Cada entrega testeada en iPhone físico, Android físico y Chrome desktop antes de presentar el hito	Todos
Onboarding iOS	Pantalla o banner que guía al usuario de	Hito 1

Criterio	Descripción	Hito
	iPhone a instalar la PWA (Safari → Compartir → Agregar a pantalla de inicio)	
Historial persistente	El usuario retoma conversaciones anteriores desde cualquier dispositivo	Hito 1
Chats múltiples	El usuario puede abrir nuevos chats, tener varios en paralelo y volver a uno anterior	Hito 1
Respuestas ancladas	Flippy no responde preguntas fuera del corpus; indica explícitamente cuando no tiene información	Hito 2
Sin citas visibles	Las respuestas no muestran referencias a documentos fuente en la interfaz	Hito 2
Análisis de imágenes	El usuario puede adjuntar una foto y Flippy la analiza cruzando con el corpus	Hito 2
Control de planes	Las funcionalidades restringidas del plan gratuito se aplican correctamente	Hito 3
Flujo de mora	Pago rechazado → banner en app → acceso restringido → reintento exitoso → acceso restaurado	Hito 3
Transferencia completa	Repositorios, Railway, Supabase y demás cuentas bajo control exclusivo del cliente al cierre	Hito 4